

Contents

1	Aliases	2
1.1	Alias Commands	2
1.2	Helpful Aliases	2
1.2.1	Git	2
1.2.2	Server	2
2	Servers	2
2.1	Connecting to Servers	2
2.1.1	SSH Key Expired	2
2.2	Moving Files	2
2.3	Running processes in background	3
2.3.1	Screens	3
2.3.2	tmux	3

1 Aliases

1.1 Alias Commands

- `alias` - Can call at any point to see a list of available Aliases
- `~/.zshrc` - Location of aliases

1.2 Helpful Aliases

1.2.1 Git

- `alias gs='git status'`
- `alias ga='git add .'`
- `alias gc='git commit .'`
- `alias gp='git push'`
- `alias gr='git pull'`

1.2.2 Server

Tip: anytime I need to run something on the server, I put the code/resources in a local folder called 'server_package', always stored in the same spot. That way I can run the aliases below to easily load those files into the server, and easily pull them back locally afterwards.

- `server() {ssh USERNAME@stat-$1.byu.edu}` - type `server g02` to join the g02 server
- `pull_server_package() {scp -r USERNAME@stat-$1.byu.edu:server_package FILE_PATH}`
 - Type `pull_server_package g02` to retrieve the 'server_package' folder from server g02
- `push_server_package() {scp -r FILE_PATH USERNAME@stat-$1.byu.edu:}`
 - Type `push_server_package g02` to send the 'server_package' folder to server g02

2 Servers

2.1 Connecting to Servers

`ssh USERNAME@stat-g01.byu.edu` - Full line for connecting to the server (just replace the number with the corresponding server number, and USERNAME with your username)

2.1.1 SSH Key Expired

`rm .ssh/known_hosts` - If you try connecting and it says "WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!", run this code to just reset all known hosts. Then try connecting again (you'll have to type 'yes' to save it as a known host again)

2.2 Moving Files

Move file from local machine to server: `scp FILE_PATH USERNAME@stat-g02.byu.edu:FILE_PATH`

Move file from server to local machine: `scp USERNAME@stat-g02.byu.edu:FILE_PATH FILE_PATH`

To move a full directory, include the recursive `-r`. Example: `scp -r rjcass13@...`

2.3 Running processes in background

2.3.1 Screens

Running a program in a screen allows it to run in the background even when you disconnect from the server

1. Login to the server
2. Start a screen session: `screen -S my_session_name`
3. Navigate to and run the desired program
4. Detach from the screen session: On a Mac: CTRL+A (nothing should change), then D (if it worked it will take you back to the original SSH session and say detached)
5. To rejoin the session, while logged in to the server, do: `screen -r my_session_name`. If you forgot what the session is called, do `screen -ls` to see all session names.
6. To end the screen session (completely kill it, no coming back), just enter `exit`. If the session is frozen and the previous method doesn't work, you can use `screen -d my_session_name` in the main SSH session.

2.3.2 tmux

Very similar to screen, running a program in a tmux allows it to run in the background even when you disconnect from the server. May not be included on all servers.

1. Login to the server
2. Start a screen session: `tmux new -s my_session_name`
3. Navigate to and run the desired program
4. Detach from the screen session: On a Mac: CTRL+B (nothing should change), then D (if it worked it will take you back to the original SSH session and say detached)
5. To rejoin the session, while logged in to the server, do: `tmux attach -t my_session`. If you forgot what the session is called, do `tmux ls` to see all tmux sessions names.
6. To end the screen session (completely kill it, no coming back), just enter `exit`. If the session is frozen and the previous method doesn't work, you can use `tmux kill-session -t session_name` in the main SSH session.